Week 1: Linear & Binary Search

Linear Search

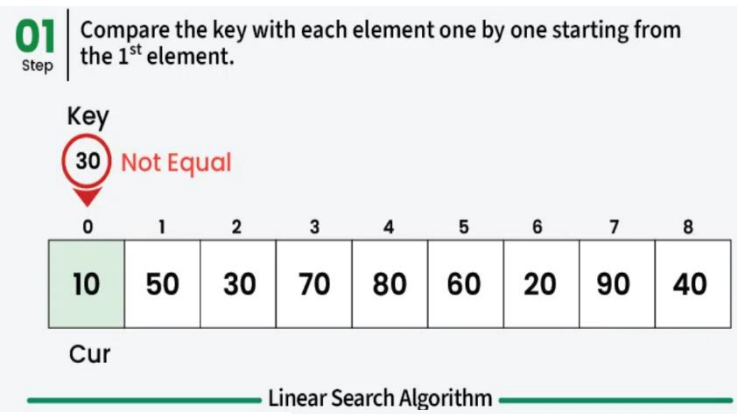
Linear Seach checks each element of a list one by one until the desired element is found or the list ends. Given an array of 'n' element 'x', find whether element 'x' is present in the array. Return the index of the first occurrence of 'x' in the array or -1 if it does not exist.

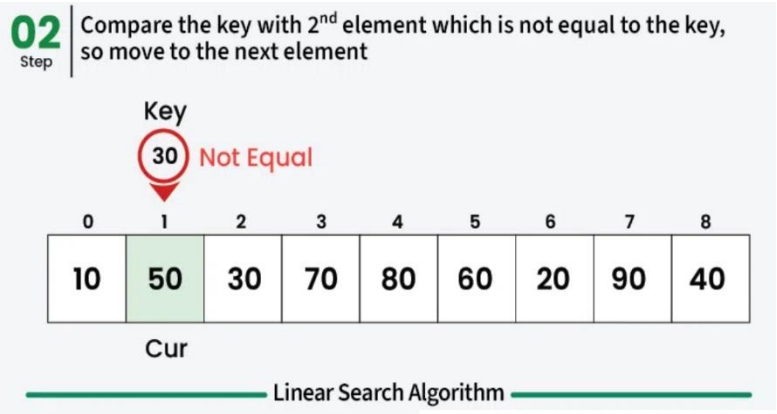
Examples

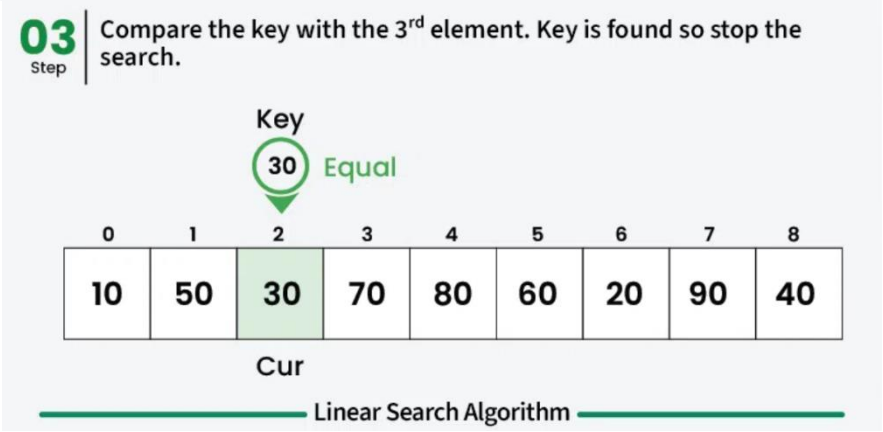
Input: `arr[] = [10, 50, 30, 70, 80, 20, 90, 40], x = 30`

Output: `2`

Explanation: For array `[10, 50, 30, 70, 80, 20, 90, 40]`, the element to be searched is 30 and it is at index 2. So, the output is `2`.







Explanation: A simple approach is to do a linear search

- Start from the leftmost element of `arr[]` and one by one compare x with each element of `arr[]`
- If x matches with an element, return the index
- If x doesn't match any of the elements return -1.

Linear Search Code:

```
def linear_search(arr, target):
    for index, value in enumerate(arr):
        if value == target:
            return index  # Target found, return its position
    return -1  # Target not found in the list
```

Linear Search Exam Questions:

1. **Basic Index Search (First Occurrence) :** Return the index of the first target match, or `-1` if the element does not exist.

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i  # Early return on first match
    return -1  # Reached only if target is not in arr
```

   Exam focus: Ensuring you place return `-1` **outside** the loop, not in an else block inside the loop.

2. **Find The Last Occurrence:** Return the index where the target appears last.

```python
# Approach 1: Traverse backward
def search_last_backward(arr, target):
    for i in range(len(arr) - 1, -1, -1):
        if arr[i] == target:
            return i
    return -1

# Approach 2: Traverse forward and overwrite
def search_last_forward(arr, target):
    last_idx = -1
    for i in range(len(arr)):
        if arr[i] == target:
            last_idx = i
    return last_idx
```

   Exam focus: Handling reverse range indexing (`range(start, stop, step)`) or maintaining a tracker variable until the loop completes.

3. **Find All Indices / Multi-Match Search**: Return a list of all indices containing the target value.

```python
def find_all_occurrences(arr, target):
    indices = []
    for i, val in enumerate(arr):
        if val == target:
            indices.append(i)
    return indices
```

   Exam focus: Correctly handling cases where the target does not exist (should return an empty list `[]`, not `None` or an error).

4. **Minimum / Maximum Value Search**: Find the extreme value (or its index) without using min() or max().

```python
def find_min_index(arr):
    if not arr:
        return -1  # Edge case: empty list

    min_idx = 0
    for i in range(1, len(arr)):
        if arr[i] < arr[min_idx]:
            min_idx = i
    return min_idx
```

   Exam focus: Initializing `min_idx = 0` (or `min_val = arr[0]`) instead of arbitrary hardcoded numbers like `0` or `999999`, which fail on all-negative lists.

5. **Conditional Search (Objects / Dictionaries)**: Searching structured data based on a specific attribute or threshold.

```python
def find_first_passing_student(students, passing_grade=60):
    for student in students:
        if student["grade"] >= passing_grade:
            return student["name"]
    return None
```

Exam focus: Dictionary key lookup and handling nested structures.

6. **2D Grid / Matrix Search**: Locate row and column coordinates `(r, c)` of a target in an unsorted matrix.

```python
def search_2d(matrix, target):
    for r in range(len(matrix)):
        for c in range(len(matrix[r])):
            if matrix[r][c] == target:
                return (r, c)
    return None
```

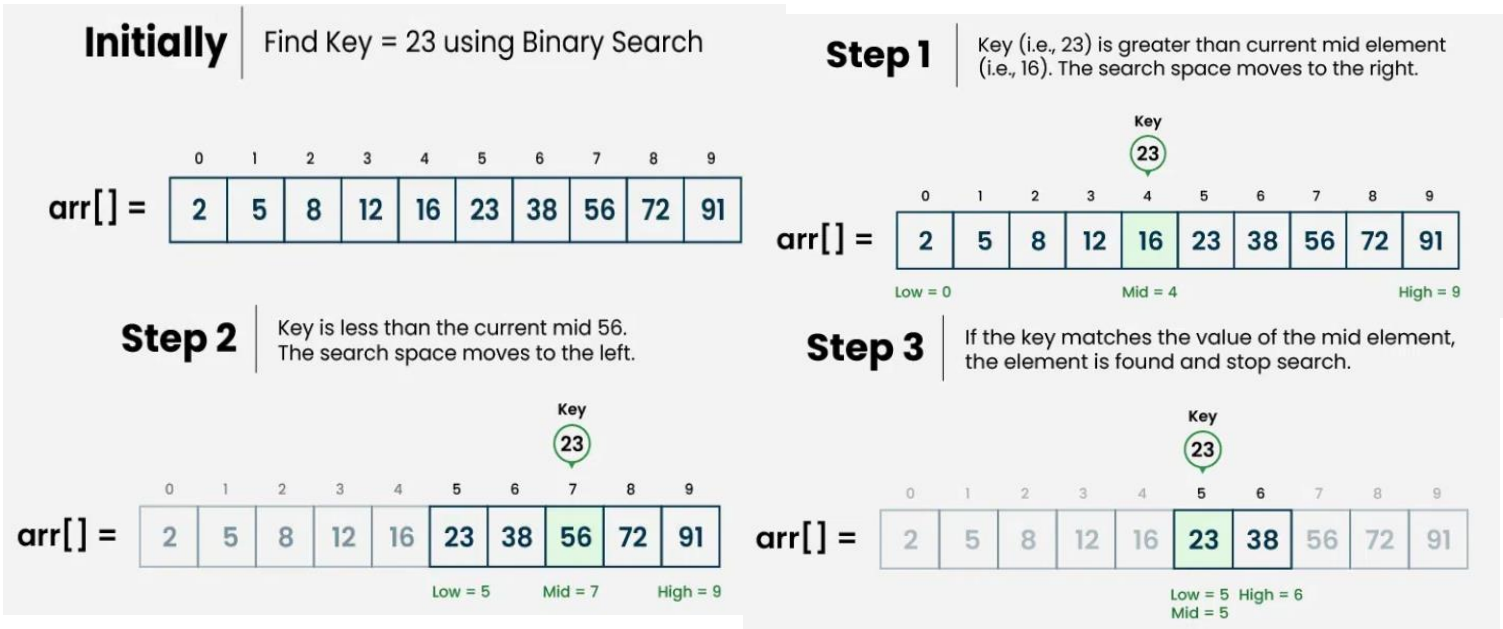
Exam focus: Nested loop execution order and time complexity analysis ($O(R \times C)$).

Binary Search

Binary Search is an efficient searching algorithm used to find an element in a **sorted array** by repeatedly dividing the search interval in half. It reduces the time complexity to O(log n), making it much faster than linear search.

Here is how Binary Search works:

1. Find the middle element of the array
2. Compare the middle element with the target key. If equal: return index.
3. If the key is smaller; search the left half
4. If the key is larger; search the right half
5. Repeat until the element is found of the search space is empty.



Binary Search Code

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1  # Target is in the right half
        else:
            high = mid - 1  # Target is in the left half

    return -1  # Target not found
```

Explanation:

- Using a while loop to implement iterative binary search.
- Initialize low and high pointers to define the search range.
- Calculates the middle index `mid = low + (high - low) // 2`.
- If `arr[mid]` is less than x, moves search to the right half.
- If `arr[mid]` is greater than x, moves search to the left half.
- If `arr[mid] == x`, returns the index.
- Returns `-1` if the element is not found.

Week 2: Timing / Benchmarking

time.perf_counter() function in Python

The `time.perf_counter()` function returns a high-resolution timer value use to measure how long a piece of code takes to run. It is designed for performance measurement, includes times spend during sleep, and is accurate for short duration.

This is example measure how much time a loop time a loop takes to execute using `perf_counter()`.

```python
from time import perf_counter

start = perf_counter()
for i in range(1000000):
    pass
end = perf_counter()
print(end - start)
```

Output: `0.05940742199982196`

Explanation:

- `perf_counter()` is called before and after the loop
- The difference `end - start` gives the execution start time in seconds.

Syntax

```
time.perf_counter()
```

Parameters: None

Examples

1. **Calculate how long a for loop takes to complete one million iterations.**

```
from time import perf_counter

t1 = perf_counter()
for i in range(1000000):
    i * i
t2 = perf_counter()
print(t2 - t1)
```

Explanation:

i. `start` stores the start time using `perf_counter()`

ii. `end - start` gives the total execution time

2. **Measures the execution time of a custom function**

```
from time import perf_counter

def calc():
    return sum(range(100000))

s = perf_counter()
calc()
e = perf_counter()
print(e - s)
```

Explanation:

i. `perf_counter()` is called before and after `calc()`

ii. The time difference shows how long the function took

3. **Shows that `perf_counter()` includes time spent during sleep.**

```
from time import perf_counter, sleep

a = perf_counter()
sleep(1)
b = perf_counter()
print(b - a)
```

Explanation:

i. `sleep(1)` pauses execution for 1 second

ii. `b - a` confirms that sleep time is included

4. Empirical Timing of Binary Search vs. Linear Search

```
def run_timing_trials(n_values, trials=100):
    for n in n_values:
        data = list(range(n))
        target = -77  # Guaranteed absent (worst case)

        # Task 1: Time linear search (target in data) across `trials` runs
        # Store total elapsed seconds in `linear_total_sec`
        start_time = time.perf_counter()
        for _ in range(trials):
            _ = target in data
        linear_total_sec = time.perf_counter() - start_time

        # Task 2: Time binary_search(data, target) across `trials` runs
        # Store total elapsed seconds in `bs_total_sec`
        start_time = time.perf_counter()
        for _ in range(trials):
            _ = binary_search(data, target)
        bs_total_sec = time.perf_counter() - start_time
```

a. Explanation: Timing Linear Search vs. Binary Search Worst-Case Performance

i. Data is sorted list from `0` to `n-1`, and `target = -77` is guaranteed absent.

ii. Looking for an absent item forces the worst-case scenario (O(n) for linear search)

iii. Running trials times averages out system CPU fluctuations.

b. Explanation: Measuring Binary Search Execution Time

i. `perf_counter()` is called before and after the search loop.

ii. The time difference gives the total seconds spent on all trials.

iii. Binary search eliminates half the remaining elements each step (O(log n)).

c. Key Observations and Scaling

i. At small sizes (`n=10`) both methods are nearly identical in speed

ii. At large sizes (`n=1,000,000`), linear search checks 1,000,000 items while binary search check only roughly 20 times

Week 3: Logs & Memory Diagrams

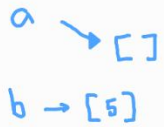
1. Count Created lists

find every [] & write down total # of lists.

line 1: a=[]
line 2: b=a
Total []: 1

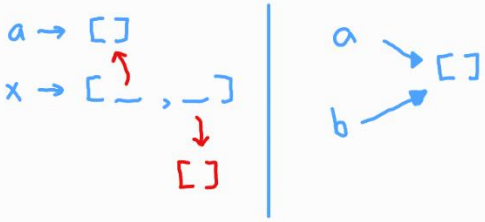
2. Draw variable reference arrows

Write each variable name (a, b, c) and draw an arrow (→) pointing to the list object.



3. Map shared references & nested elements

Draw arrows (→) showing aliases (multiple variables pointing to the exact same list) and arrows leading from index slots to nested inner lists



4. Trace mutations vs. reassignments

a) In place mutation (.append())

modify or add items directly inside existing list box →

old: a → []
new (append): a → [30]

b) Reassignments (e.g. a=[], a=b)

draw brand new list object and redirect a's arrows to the new target →



Q1

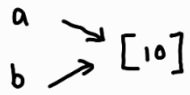
```
a = [10]
b = a

b.append(20)

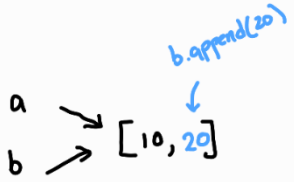
print(a)
print(b)
```

How many Lists: 1

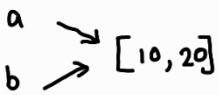
Setup Variables & Reference arrows:



Trace Mutations / Index Reassignment:



Final memory diagram:



predicted output:

a: [10, 20]

b: [10, 20]

Q2

```
a = []
x = [a, []]

a.append(10)
x[1].append(20)

print(x)
```

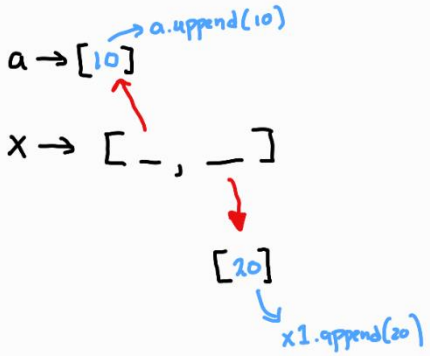
How many Lists: 3

Setup Variables & Reference arrows:

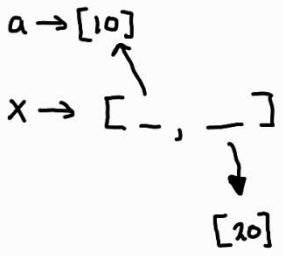
a → [ ]

x → [_ , _]

Trace mutations / Index Reassignment:



Final memory diagram:



predicted output:

a: [10]

x: [[10], [20]]

Q3

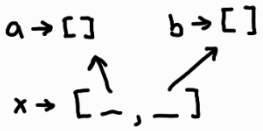
```
a = []
b = []
x = [a, b]

a.append(b)
b.append(5)

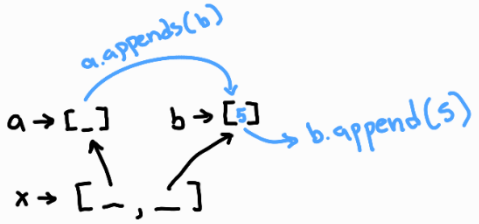
print(a)
print(b)
print(x)
```

How many Lists: 3

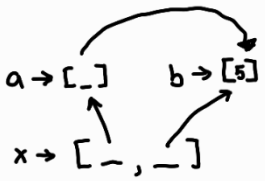
Setup Variables & Reference arrows:



Trace Mutations / Index Reassignment:



Final memory diagram:



predicted output:

a: [[5]]

x: [[[5]], [5]]

Q4

```
a = []
b = [a]
c = a

a = [99]
c.append(10)

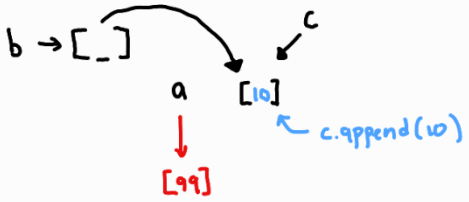
print(a)
print(b)
print(c)
```

How many Lists: 3

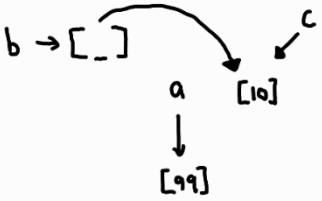
Setup Variables & Reference arrows:



Trace Mutations / Index Reassignment:



Final memory diagram:



predicted output:

a : [99]

b : [[10]]

c : [10]

Q5

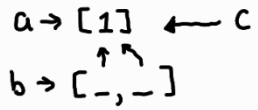
```
a = [1]
b = [a, a]
c = a

c.append(2)
b[0] = []

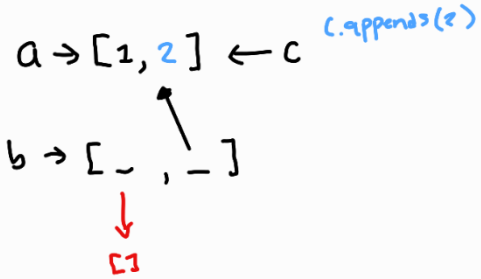
print(a)
print(b)
print(c)
```

How many Lists: 3

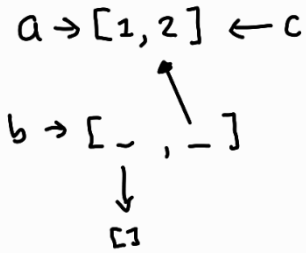
Setup Variables & Reference arrows:



Trace Mutations / Index Reassignment:



Final memory diagram:



predicted output:

a: [1,2]

b: [[], [1,2]]

c: [1,2]

Week 4: Big O Notation

Big O notation is used to describe the time or space complexity of algorithms. Big-O is a way to express an upper bound of an algorithm's time or space complexity.

- Describes the asymptotic behavior (order of growth of time or space in terms of input size) of a function, not its exact value.
- Can be used to compare the efficiency of different algorithms or data structures.
- It provides an **upper limit** on the time taken by an algorithm in terms of the size of the input. We mainly consider the worst case scenario of the algorithm to find its time complexity in terms of Big O

Concept	Meaning
Input size n	Number of elements you're processing (len of list, digits, nodes…)
Growth Rate	How cost scales when n doubles
Dominant term	The term that wins as n → ∞ (e.g. n^2 beats n)
Worst case	The maximum work over all inputs of size n
Amortized	Average cost over many operations (e.g. list append)

The Two Golden Rules

Rule 1 — Drop constants: O(2n) = O(n), O(100n^2) = O(n^2). Constants matter for real speed but not for scaling class.

Rule 2 — Drop lower-order terms: O(n^2 + n + 3) = O(n^2). The fastest-growing term dominates.

```
def f(n):
    total = 0
    for i in range(n):      # O(n)
        total += i
    for i in range(n):      # O(n)
        for j in range(n):  # O(n²)
            total += i * j
    return total
# Total = O(n + n²) → O(n²)   ← drop the n
```

The Complexity Ladder (Fast → Slow)

Notation	Name	Example	N=1000 work	N doubles → cost
O(1)	Constant	`dict[key], list[i]`	1	unchanged
O(log n)	Logarithmic	Binary search	~10	+1 Step
O(n)	Linear	Single loop, max()	1,000	× 2
O(n log n)	Linearithmic	sorted()	~10,000	~2.2x
O(n^2)	Quadratic	Nested loops, bubble sort	1,000,000	× 4
O(2^n)	Exponential	Subsets, naive Fibonacci	2^{1000}	Squares
O(n!)	Factorial	Permutations, brute-force TSP	1000!	explodes

The Python-Specific Trap: Built Costs

Operation	Complexity	Why
`lst[i], lst[-1]`	**O(1)**	direct index
`append(x)`	**O(1) amortized**	occasional resize, averages out
`pop() (last)`	**O(1)**	remove tail
`pop(i) / insert(i, x)`	**O(n)**	everything shifts
`x in lst`	**O(n)**	linear scan
`len(lst)`	**O(1)**	stored
`lst[a:b] (slice)`	**O(k) where k = b−a**	copies
`lst.sort() / sorted(lst)`	**O(n log n)**	Timsort
`lst.index(x), lst.count(x)`	**O(n)**	scan
`lst.extend(other)`	**O(k)**	k = len(other)

Operation	Complexity
`len(s), s[i]`	O(1)
`sub in s, s.find(sub)`	O(n)
`s + t`	O(n) — creates a new string
`s += c in a loop`	O(n²) ← builds a new string every time
`s[a:b]`	O(n)

Operation	Complexity
`append, appendleft, pop, popleft`	O(1)
`d[i]` (random index)	O(n)

Three Python Gotchas

```
# GOTCHA 1 — prepend to a list is O(n) per op → O(n²) total
lst = []
for i in range(n):
    lst.insert(0, i)          # BAD: O(n²)
# FIX: use deque
from collections import deque
dq = deque()
for i in range(n):
    dq.appendleft(i)          # GOOD: O(n)

# GOTCHA 2 — membership test: list is O(n), set is O(1)
nums = list(range(n))
if x in nums: ...             # O(n)
s = set(nums)
if x in s: ...                # O(1) average

# GOTCHA 3 — building a string with += is O(n²)
result = ""
for c in text:
    result += c               # BAD: O(n²)
result = "".join(text)        # GOOD: O(n)
```

Worked Examples

Example A — Linear search → O(n)

```
def linear_search(lst, target):
    for i, x in enumerate(lst):   # one pass
        if x == target:
            return i
    return -1
# Worst case: target not present → n checks → O(n)
```

Example B — Binary search → O(log n)

```
def binary_search(lst, target):
    lo, hi = 0, len(lst) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if lst[mid] == target: return mid
        elif lst[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
# Search space halves each loop → ~log2(n) steps → O(log n)
```

Example C — Nested loop → O(n²)

```
def all_pairs(lst):
    for i in range(len(lst)):       # n times
        for j in range(len(lst)):   # n times
            print(lst[i], lst[j])   # total n × n = n²
```

Example D — Count duplicates: list O(n²) vs set O(n)

```
def has_dup_list(lst):
    for i in range(len(lst)):
        if lst[i] in lst[i+1:]:     # each `in` is O(n) → O(n²)
            return True
    return False

def has_dup_set(lst):
    seen = set()
    for x in lst:
        if x in seen:               # O(1) each → O(n) total
            return True
        seen.add(x)
    return False
```

Example E — Naive Fibonacci → O(2ⁿ)

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)   # each call spawns 2 → ~2^n calls
```

Pattern	Space
Variables only	O(1)
One new list of size n	O(n)
Nested copies	O(n²)
Recursion depth	O(depth) — also watch Python's ~1000 recursion limit

```python
def double(lst):
    return [x * 2 for x in lst]   # O(n) extra space
```

How to Look at Code and Determine Big O – A Repeatable Method

The 6-Step Procedure

Step 1 – Find n (the input size)

Ask: "What am I looping over?" Usually it's the length of a list/string/dict, a number of nodes, etc.

```python
def f(lst):        # n = len(lst)
    ...
def g(n):          # n is literally the number
    ...
```

Step 2 – Circle every "work unit"

A work unit is anything that scales: a `for` loop, a `while` loop, a recursive call, or a built-in that scans (`in` on a list, `max`, `sorted`, slicing).

Step 3 — For each loop, count iterations

Code shape	Iterations	Class
`for i in range(n)`	n	O(n)
`for x in lst`	len(lst)	O(n)
`while i < n: i += 1`	n	O(n)
`while n > 1: n //= 2`	$\log_2 n$	O(log n)
`for i in range(n): i *= 2`	$\log_2 n$	O(log n)

The one tell to memorize: counter doubles or halves → log n. Counter +1 / −1 → n.

Step 4 — Look inside the loop

Any built-in call or inner loop has its own cost. Multiply them.

```python
for x in lst:          # O(n) outer
    sorted(lst)        # O(n log n) inside
# Total = O(n × n log n) = O(n² log n)
```

Step 5 — Combine

- Sequential blocks (one after another) → ADD, then keep the biggest.
- Nested (one inside another) → MULTIPLY.

```
# sequential:
for i in range(n): ...        # O(n)
for i in range(n):            # O(n)
    for j in range(n): ...    #   → O(n²)
# Total = O(n + n²) = O(n²)   (add → keep dominant)
```

Step 6 — Drop constants & lower-order terms

O(2n) → O(n) · O(n² + 5n + 7) → O(n²) · O(3) → O(1)

The "Recognize the Shape" Cheat Table

You see this	Say this	Big O
Plain statement, `d[k]`, `lst[i]`, arithmetic	constant	O(1)
One for/while over n	linear	O(n)
Two loops back-to-back	still linear	O(n)
Loop inside a loop (both ~n)	quadratic	O(n²)
Three nested loops	cubic	O(n³)
Counter doubles/halves	logarithmic	O(log n)
`sorted()` / `.sort()`	linearithmic	O(n log n)
Loop calls `sorted()` each time	n² log n	O(n² log n)
`x in list` inside a loop	quadratic	O(n²)
`x in set` inside a loop	linear	O(n)
Recursion that calls itself twice	exponential	O(2ⁿ)
Recursion that calls itself once, halving	logarithmic	O(log n)

Worked Example

```
def mystery(lst):
    n = len(lst)                    # 1. n = len(lst)
    total = 0                       # O(1)
    for x in lst:                   # 2. loop → runs n times → O(n)
        total += x                  #    O(1) inside
    for i in range(n):              # 3. loop → n times
        for j in range(n):          #    nested → n times
            total += lst[i] * lst[j] #   O(1) inside → n × n = n²
    return total
```

Thinking out loud:

1. "First loop: n iterations, constant work → O(n)."
2. "Second loop: n iterations, and inside it another n → O(n²)."
3. "They're sequential → add: O(n) + O(n²)."
4. "Drop lower term → O(n²)." ✅

```python
def find_duplicates(lst):
    for i in range(len(lst)):       # n iterations
        if lst[i] in lst[i+1:]:     # ← `in` on a list is O(n)!
            return True             #    plus slicing is O(k)
    return False
```

The trap: the inner line looks like "one check," but lst[i] in lst[i+1:] scans the rest of the list → O(n) each iteration.

- Outer O(n) × inner O(n) → O(n²).

```python
def find_duplicates_fast(lst):
    seen = set()                    # O(1)
    for x in lst:                   # n iterations
        if x in seen:               # `in` on a SET is O(1)
            return True
        seen.add(x)                 # O(1)
    return False
# O(n) total
```

The 5 Most Common Mistakes

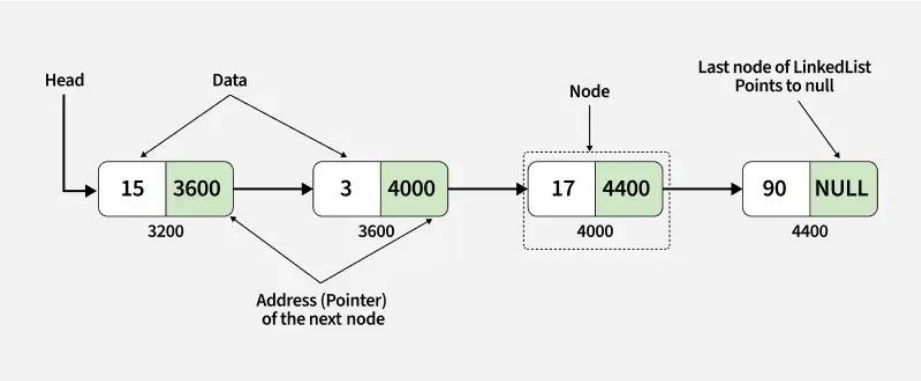
Mistake	Fix
Counting two sequential loops as n²	They add → still O(n)
Forgetting in list / max() / slicing scan	They're O(n) each
Thinking in set/in dict is O(n)	It's O(1) average
Counting string += c in a loop as O(n)	It's O(n²) — use "".join()
Forgetting the nested loop multiplies	n × n = n², not n + n

Week 5: OOP & Linked Lists

Linked List

A linked list is a type of linear data structure individual items are not necessarily at contiguous locations. The individual items are called nodes and connected with each other using links.

- A node contains two things first is data and second is a link that connects it with another node.
- The first node is called the head node, and we can traverse the whole list using this head and next links.



Creating Node with Class

We have created a Node class in which we have defined a `__init__` function to initialize the node with the data passed as an argument and a reference with None because if we have only one node then there is nothing in its reference.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Implementation of Simple 4 Nodes Linked List

Example: Below is a simple example to create a singly linked list with three nodes containing integer data.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Create nodes
node1 = Node(15)
node2 = Node(3)
node3 = Node(17)
node4 = Node(90)

# Link nodes
node1.next = node2
node2.next = node3
node3.next = node4

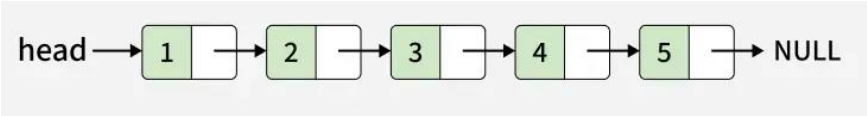
head = node1  # Head points to the first node

# Traverse and print the linked list
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Output: 15 -> 3 -> 17 -> 90 -> None

Singly Linked List

A singly linked list is a fundamental data structure; it consists of nodes where each node contains a data field and a reference to the next node in the linked list. The next of the last node is null, indicating the end of the list. Linked Lists support efficient insertion and deletion operations.



In a singly linked list, each node consists of two parts: data and a pointer to the next node. This structure allows nodes to be dynamically linked together, forming a chain-like sequence.

```
# Definition of a Node in a singly linked list
class Node:
    def __init__(self, data):

        # Data part of the node
        self.data = data
        self.next = None
```

In this example, the Node class contains an integer data field (**data**) to store the information and a pointer to another Node (**next**) to establish the link to the next node in the list.

Creating an Example Linked List of Size 3 to Understand Working

Create the first node

- Allocate memory for the first node and Store data in it.
- Mark this node as head.

Create the second node

- Allocate memory for the second node and Store data in it.
- Link the first node's next to this new node.

Create the third node

- Allocate memory for the third node and Store data in it.
- Link the second node's next to this node.
- Set its next to NULL to ensure that the next of the last is NULL.

```
# constructor to initialize a new node with data
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None

# Create the first node (head of the list)
head = Node(10)

# Link the second node
head.next = Node(20)

# Link the third node
head.next.next = Node(30)

# Link the fourth node
head.next.next.next = Node(40)

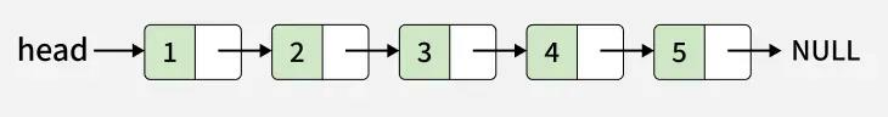
# printing linked list
temp = head
while temp is not None:
    print(temp.data, end=" ")
    temp = temp.next
```

Output: 10 20 30 40

Traversal of LL

Traversal of Singly Linked List is one of the fundamental operations, where we traverse or visit each node of the linked list. In this article, we will cover how to traverse all the nodes of a singly linked list along with its implementation.

Examples:



Output: *1 -> 2 -> 3 -> 4 -> 5*

Explanation: *Every element of each node from head node to last node is printed which means we have traversed each node successfully*

```
# a linked list node
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# function to traverse and print the singly linked list
def traverseList(head):
    # base condition is when the head is None
    if head is None:
        print()
        return

    # printing the current node data
    print(head.data, end="")

    # print arrow if not the last node
    if head.next is not None:
        print(" -> ", end="")

    # moving to the next node
    traverseList(head.next)


if __name__ == "__main__":
    # create a hard-coded linked list:
    # 10 -> 20 -> 30 -> 40
    head = Node(10)
    head.next = Node(20)
    head.next.next = Node(30)
    head.next.next.next = Node(40)

    traverseList(head)
```

Output: 10 -> 20 -> 30 -> 40

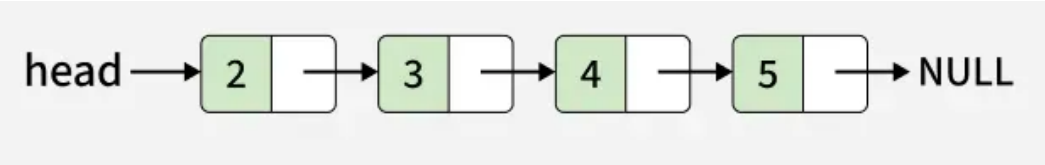
Time Complexity: O(n), where n is the number of nodes in the linked list.

Insertion of LL at the FRONT

Given a head of the linked list, Insert a new node at the beginning/start/front of the linked list.

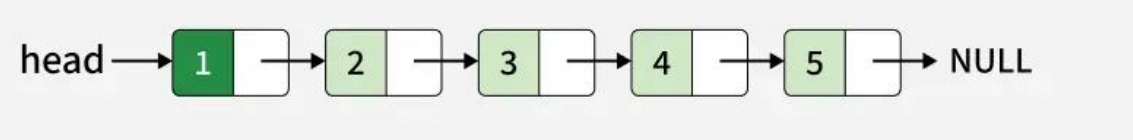
Example:

Input: *x = 1*



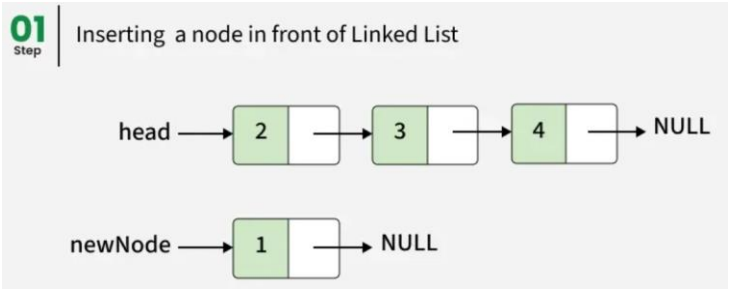
Output: *1 -> 2 -> 3 -> 4 -> 5*

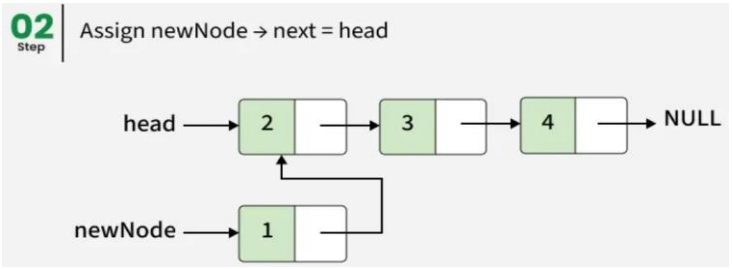
Explanation: *We can see that 1 is inserted at the beginning of the linked list.*

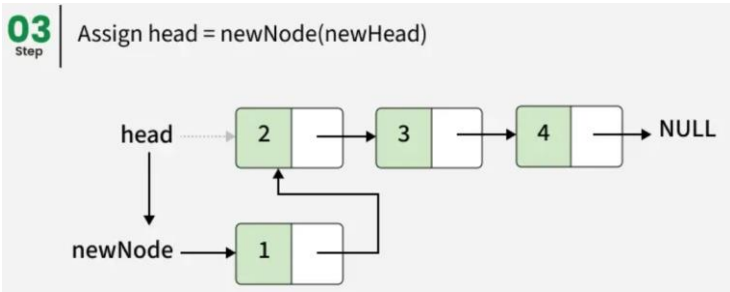


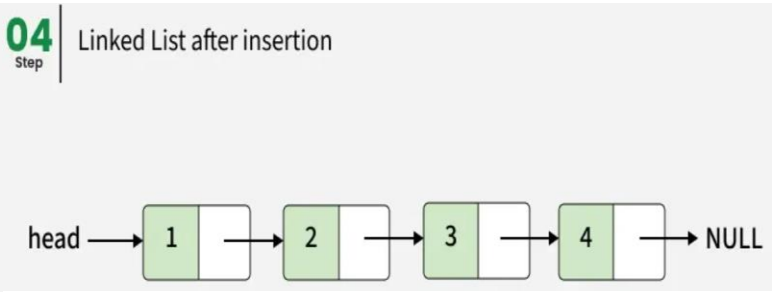
Approach:

To insert a new node at the front, we create a **new node** and point its **next** reference to the **current head** of the linked list. Then, we update the **head** to be this **new node**. This operation is efficient because it only requires adjusting a few pointers.









Algorithm:

- Make the first node of Linked List linked to the new node
- Remove the head from the original first node of Linked List
- Make the new node as the Head of the Linked List.

```
class Node:
def __init__(self, x):
   # Store the actual value passed into the node
   self.data = x

   # When a node is first created, it points to nothing (None)
   # because it is not yet connected to any other node.
   self.next = None



# insertAtFront: Prepends a new node to the start of the linked list.
def insertAtFront(head, x):
   # Step 1: Create a brand new node with value 'x'
   # At this moment: newNode -> [x | None]
   newNode = Node(x)

   # Step 2: Link the new node to the existing chain
   # The new node's 'next' pointer now references the current head node.
   # If the list was empty (head is None), newNode.next simply becomes None.
   newNode.next = head

   # Step 3: Return the new node, which now serves as the updated head of the list
   return newNode

def printList(head):
   # Initialize a traversal pointer starting at the head of the list.
   # We avoid modifying 'head' directly so the caller retains the list entry point.
   curr = head

   # Loop until 'curr' runs off the end of the list (points to None)
   while curr is not None:
     # Print the current node's value without adding a newline
     print(curr.data, end="")
     # Format output: print an arrow only if another node follows
     if curr.next is not None:
        print(" -> ", end="")
     # Move the pointer to the next consecutive node in the chain
     curr = curr.next

   # Print a trailing newline once the full list has been output
   print()
```

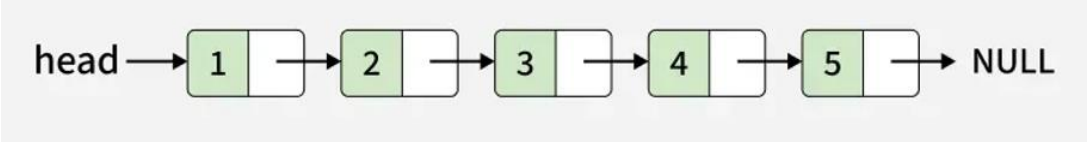
Output: 1 -> 2 -> 3 -> 4 -> 5

Time Complexity: O(1), We have a pointer to the head and we can directly attach a node and update the head pointer. So, the Time complexity of inserting a node at the head position is O(1).

Insertion of LL at the END

Given a head of the **linked list**, we need to insert a new node at the end of the linked list.

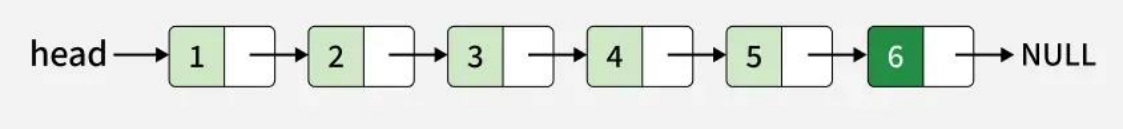
Example:



Input: *x = 6*

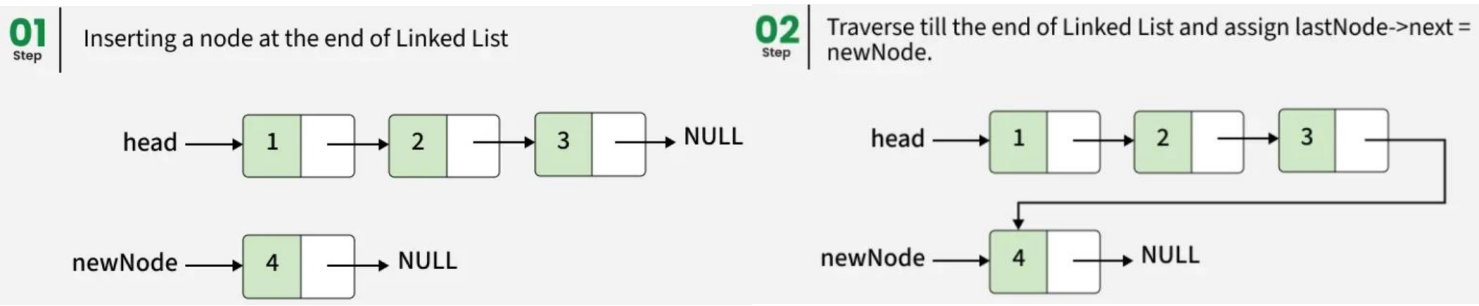
Output: *1 -> 2 -> 3 -> 4 -> 5 -> 6*

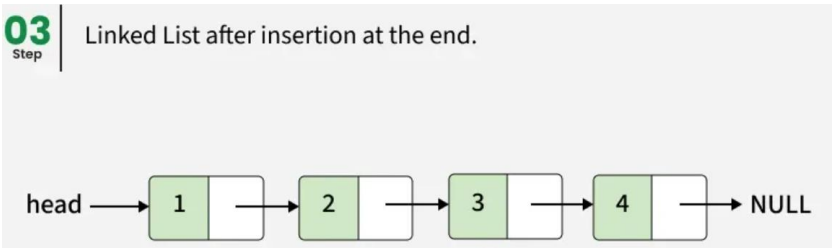
Explanation: *We can see that 6 is inserted at the end of the linkedlist**.***



Approach:

*Inserting at the end involves traversing the entire list until we reach the last node. We then set the **last node's** next reference to point to the **new node**, making the **new node** the last element in the list.*





Following is the approach to add a new node at the end of the linked list:

- Create a new node and set its next pointer as NULL since it will be the last node.
- Store the head reference in a temporary variable
- If the Linked List is empty, make the new node as the head and return
- Else traverse till the last node
- Change the next pointer of the last node to point to the new node

```
# Given the head of a list and an int, appends
# a new node at the end and returns the head.
def insertAtEnd(head, x):
    # Create a new node
    newNode = Node(x)

    # If the Linked List is empty, make
    # the new node as the head and return
    if head is None:
        return newNode

    # Store the head reference in a temporary variable
    last = head

    # Traverse till the last node
    while last.next is not None:
        last = last.next

    # Change the next pointer of the last node
    # to point to the new node
    last.next = newNode

    # Return the head of the list
    return head
```

Output: 1 -> 2 -> 3 -> 4 -> 5 -> 6

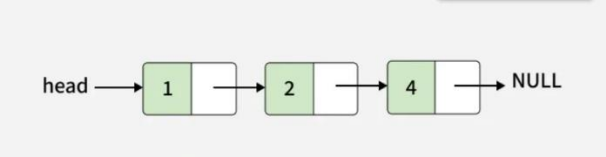
Time Complexity: O(n) where n is the length of the linked list

Insertion of LL at a SPECIFIC POSITION

Given a singly linked list, a position **pos**, and **val**, Insert that data into a linked list at the given position.

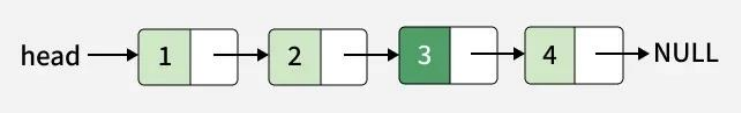
Example:

***Input**: val = 3, pos = 3*



***Output**: 1 -> 2 -> 3 -> 4*

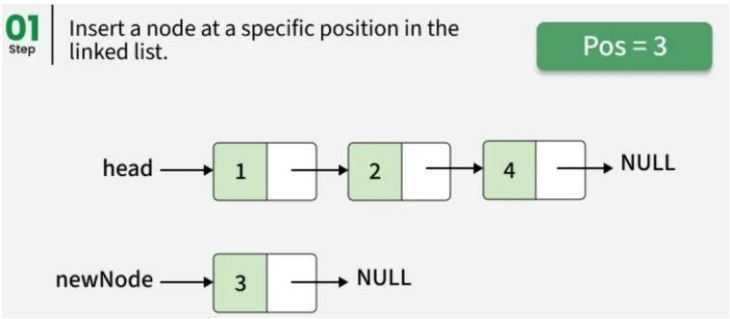
***Explanation:** Node inserted at position 3.*

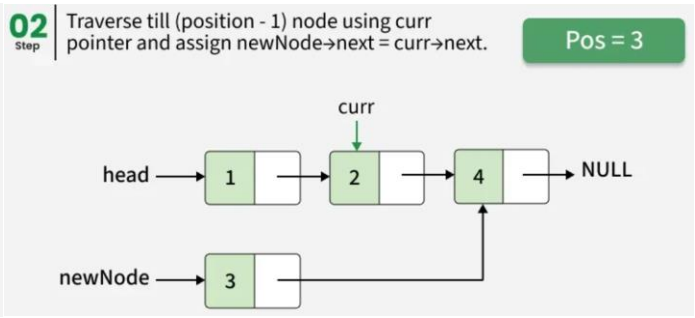


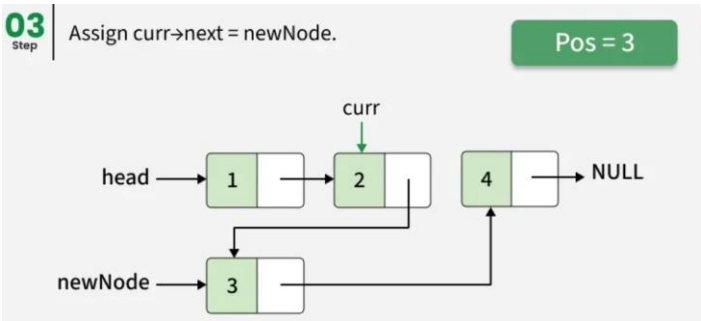
Iterative Method - O(n) time:

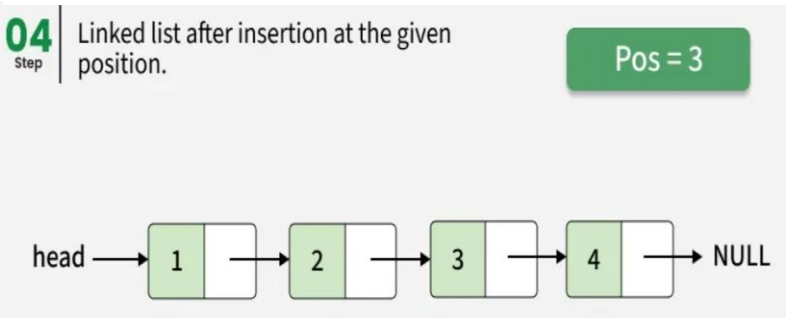
The idea is to create a new node, traverse to the node just before the given position, and update the links to insert the new node into the list.

- Initialize curr to head and create a new node with the given val.
- Traverse the linked list until curr reaches the node just before the given position.
- Set the new node's next link to curr's next node.
- Update curr's next link to the new node.
- Return the head of the modified linked list.









Code Implementation:

```
def insertPos(head, pos, val):

    if pos < 1:
        return head

    # Head will change if pos = 1
    if pos == 1:
        newNode = Node(val)
        newNode.next = head
        return newNode

    curr = head

    # Traverse to the node just before the given position
    for _ in range(1, pos - 1):
        if curr is None:
            break
        curr = curr.next

    # If position is greater than the number of nodes
    if curr is None:
        return head

    newNode = Node(val)

    # Update the links
    newNode.next = curr.next
    curr.next = newNode

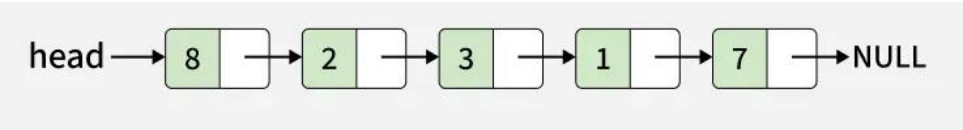
    return head
```

Output: 1 -> 2 -> 3 -> 4

Deletion of LL from FRONT

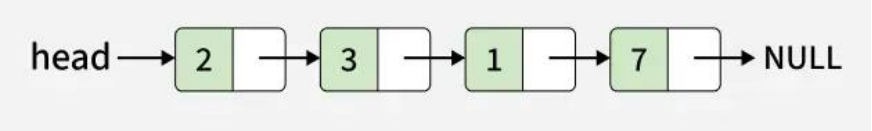
Given a head of the **linked list**, we need to **remove** the first node from the given **linked list**.

Example:



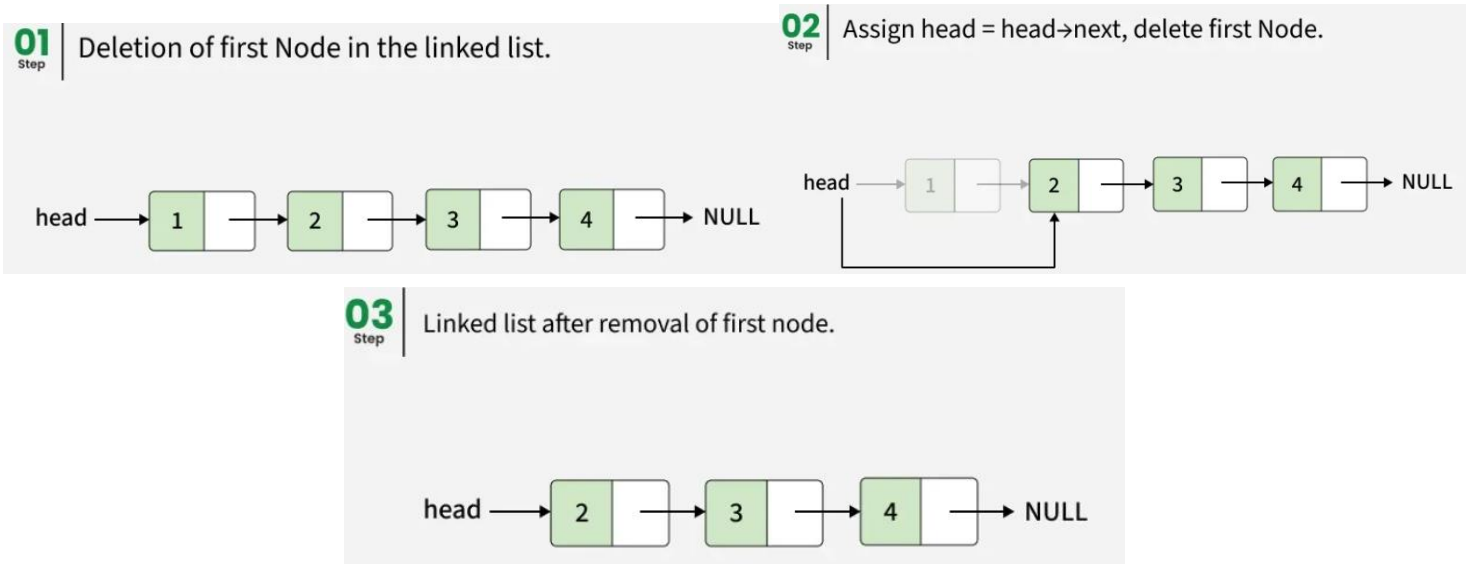
Output: *2 -> 3 -> 1 -> 7 -> NULL*

Explanation: *After deleting head from the given linked list, we'll be left with just 2 -> 3 -> 1 -> 7.*



By Shifting head node to next node of head - O(1) Time and O(1) Space:

- *To remove the first node of a **linked list**, store the current **head** in a temporary variable **(temp)**, move the **head** pointer to the next node, **delete** the temporary **head** node and finally , return the new **head** of the linked list.*



Code Implementation:

```
# Delete the head node and return the new head
def deleteHead(head):

    # Check if the list is empty
    if head is None:
        return None

    # Store the current head in a
    # temporary variable
    temp = head

    # Move the head pointer to the next node
    head = head.next

    # Free the memory of the old head node
    # (Python garbage collector will handle it)
    temp = None

    return head
```

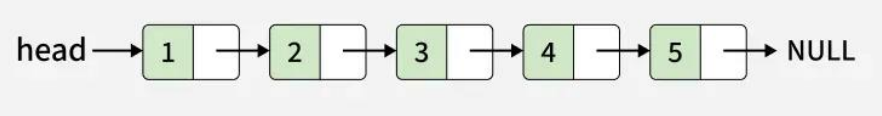
Time Complexity: O(1), because the operation to delete the head node is performed in constant time.

Space Complexity: O(1)

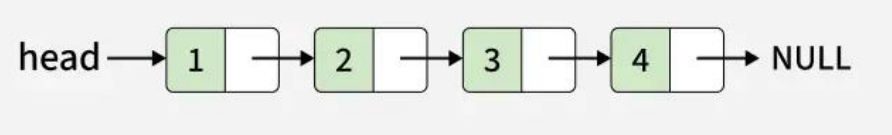
Deletion of LL from END

Given the head of a linked list, delete the last node of the given linked list.

Example:



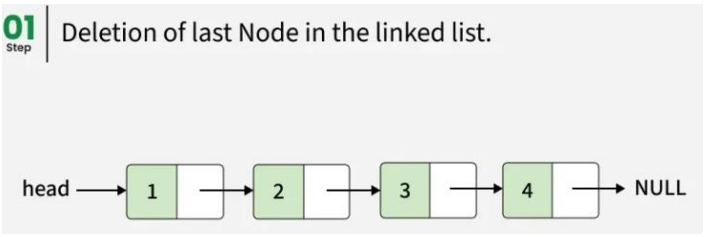
Output:

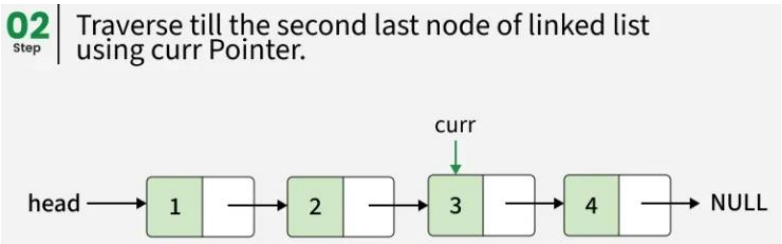


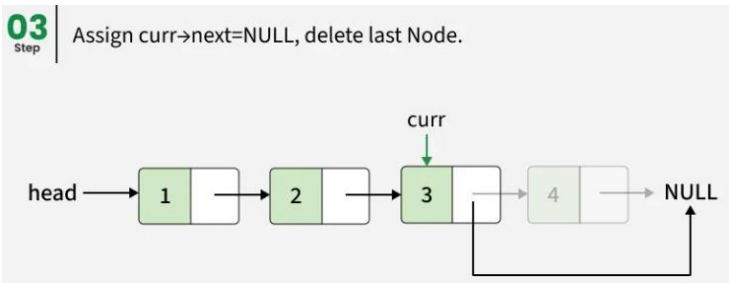
Explanation: *The last node of the linked list is 5, so 5 is deleted.*

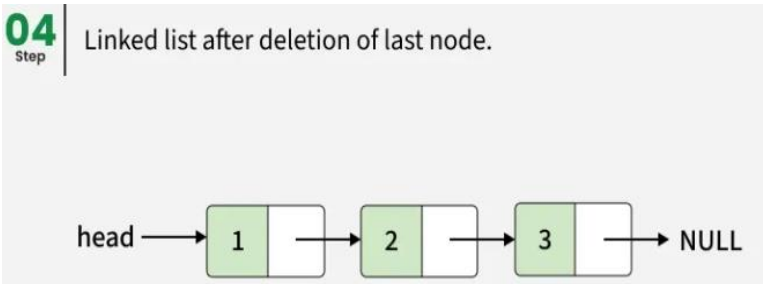
Approach:

- *To perform the deletion operation at the end of linked list, we need to traverse the list to find the* ***second last node****, then set its next pointer to* ***null****. If the list is empty then there is no node to delete or has only one node then point* ***head*** *to* ***null****.*









Step-by-step approach:

- Check if list is empty then return **NULL**.
- If the list has only one node then delete it and return **NULL**.
- Traverse the list to find the second last node.
  => Set the next pointer of **second last** node to **NULL**.

Implementation:

```
# Function to remove the last
# node of the linked list
def removeLastNode(head):
    # If the list is empty, return None
    if head is None:
        return None

    # If the list has only one node,
    # delete it and return None
    if head.next is None:
        return None

    # Find the second last node
    secondLast = head
    while secondLast.next.next is not None:
        secondLast = secondLast.next

    # Delete the last node by making
    # second_last point to None
    secondLast.next = None
    return head
```

Output: 1 -> 2 -> 3 -> 4 -> nullptr

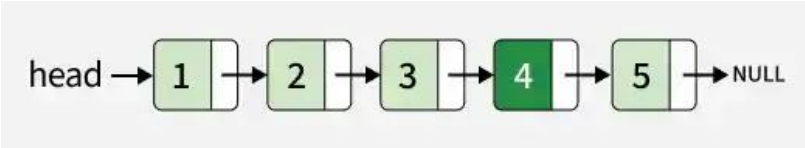
Time Complexity: O(n), traversal of the linked list till its end, so the time complexity required is *O(n)*.

Deletion of LL from SPECIFIC POSITION

Given the **head** of a linked list and an integer **x**, delete the node at position x and return the updated head of the linked list.
Note: Positions use 1-based indexing.

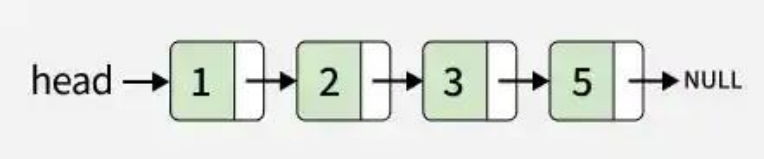
Example:

- ***Input:*** *x = 4,*



Output: *1 -> 2 -> 3 -> 5*

Explanation: *After deleting the node at the 4th position, the linked list is as*



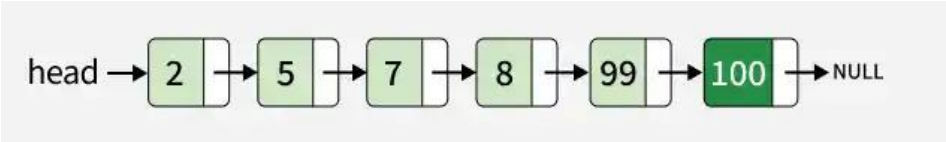
Single Traversal Deletion - O(n) Time and O(1) Space:

- The idea is to traverse the linked list only once while keeping track of the previous node. When the node at position x is reached, update the previous node's next pointer to skip it and delete the node. If x is 1, simply update the head pointer.

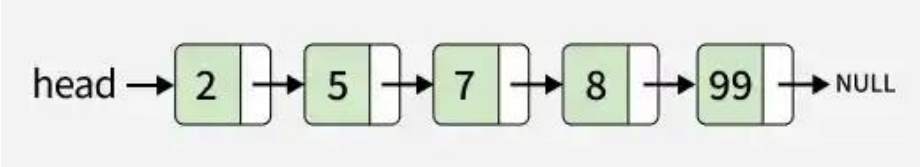
Working of Approach:

- If x == 1, delete the head node.
- Traverse the list while maintaining both current and previous pointers.
- Stop when the current node reaches position x.
- Update the previous node's next pointer.
- Delete the current node and return the updated head.

Input: x = 6,



- The linked list is 2 -> 5 -> 7 -> 8 -> 99 -> 100 and x = 6, so the head is not deleted.
- Traverse the list while updating prev and temp. After the loop, prev points to 99 and temp points to 100.
- Update prev->next to temp->next (nullptr), removing 100 from the linked list.
- Delete the node pointed to by temp.
- The updated linked list becomes



Code Implementation:

```
def deleteNode(head, x):
  temp = head

  # Case 1: Head is to be deleted
  if x == 1:
    head = temp.next
    del temp
    return head

  # Case 2: Traverse to the node
  # before the one to be deleted
  prev = None
  for i in range(1, x):
    prev = temp
    temp = temp.next

  # Delete the node at position x
  prev.next = temp.next
  del temp
```

```
    return head
```

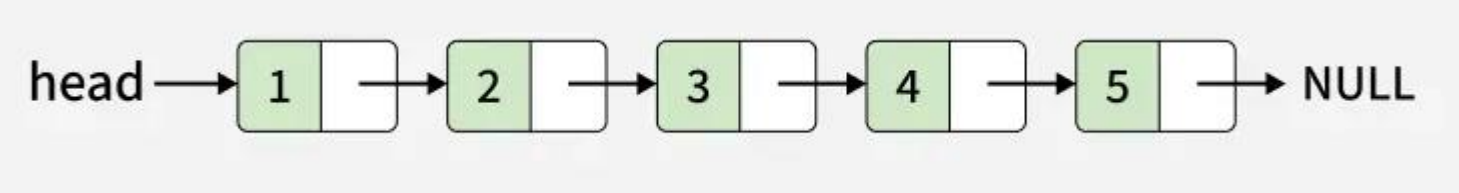
Output: 2 -> 5 -> 7 -> 8 -> 99

Searching of LL

Given the **head** of a singly linked list and an integer key, check if the key is present in the linked list or not.

Example:

Input: *key = 5*



Output: *true*

Explanation: *5 is present in the linked list.*

Iterative Approach - O(n) Time and O(1) Space:

- The idea is to traverse the linked list starting from the head. At each node, check if its value matches the given key; if it does, return true. Otherwise, move to the next node and continue the process until the end of the list. If the key is not found after the traversal, return false.

```
# checks whether key is present in linked list
def search_key(head, key):

    # initialize curr with the head of linked list
    curr = head

    # iterate over all the nodes
    while curr is not None:

        # if the current node's value is equal to key,
        # return true
        if curr.data == key:
            return True

        # move to the next node
        curr = curr.next

    # if there is no node with value as key, return false
    return False
```

Output: true

Updating/Modifying of LL

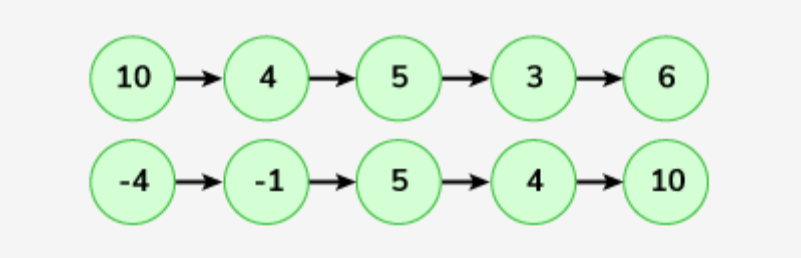
Given the head of a singly linked list, modify the list by pairing the first node with the last node, the second node with the second last node, and so on. For each pair:

- Replace the value of the first node with (value of the second node − value of the first node).
- Replace the value of the second node with the original value of the first node.

If the linked list has an odd number of nodes, the middle node remains unchanged. Return the head of the modified linked list.

Example:

- ***Input:*** *10 -> 4 -> 5 -> 3 -> 6*
  Output: *-4 -> -1 -> 5 -> 4 -> 10*



Explanation*:*
The linked list has an odd number of nodes, so the middle node (5) remains unchanged.
Pair (10, 6): Update the first node to 6 - 10 = -4, and the last node becomes the original value of the first node, 10.
Pair (4, 3): Update the second node to 3 - 4 = -1, and the second last node becomes the original value of the second node, 4.

Reverse the Second Half of the Linked List - O(n) Time and O(1) Space:

- The idea is to avoid using extra space by modifying the linked list directly. Find the middle of the list, reverse the second half, and process both halves simultaneously to update the paired nodes. Finally, reverse the second half again to restore the original structure of the linked list.

Working of Approach:

- Use slow and fast pointers to find the middle of the linked list.
- Reverse the second half of the linked list.
- Traverse the first half and the reversed second half together, updating the node values according to the given rules.
- Reverse the second half again and reconnect it to restore the original list structure.
- Return the head of the modified linked list.

```
def reverse(head):
    prev = None
    curr = head

    while curr is not None:
        next = curr.next
        curr.next = prev
        prev = curr
        curr = next

    return prev


def modifyTheList(head):
    if head is None or head.next is None:
        return head

    slow = head
    fast = head

    # Find the middle of the linked list
    while fast.next is not None and fast.next.next is not None:
        slow = slow.next
        fast = fast.next.next

    secondHalf = slow.next
    slow.next = None

    # Reverse the second half
    secondHalf = reverse(secondHalf)

    first = head
    second = secondHalf

    # Modify paired node values
    while second is not None:
        value = first.data
        first.data = second.data - value
        second.data = value

        first = first.next
        second = second.next

    # Restore the original structure
    slow.next = reverse(secondHalf)

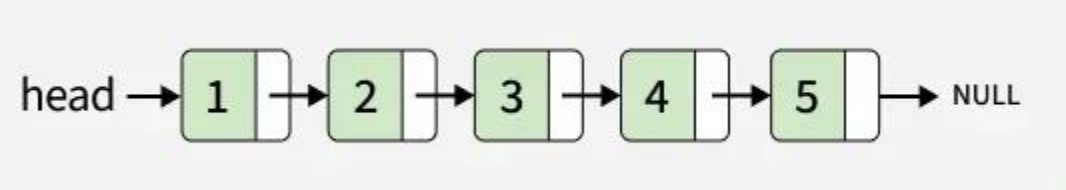
    return head
```

Output: -4 -> -1 -> 5 -> 4 -> 10

Reversal of LL

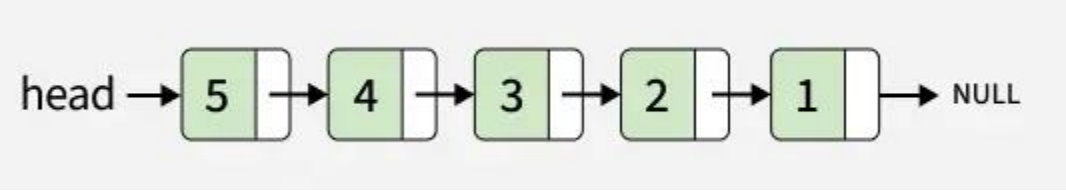
Given the head of a linked list, reverse the list and return the new head.

Example:



***Output**: 5 -> 4 -> 3 -> 2 -> 1*

***Explanation:** After reversing the linked list*

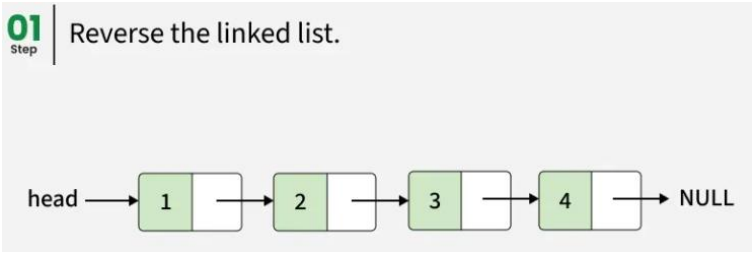


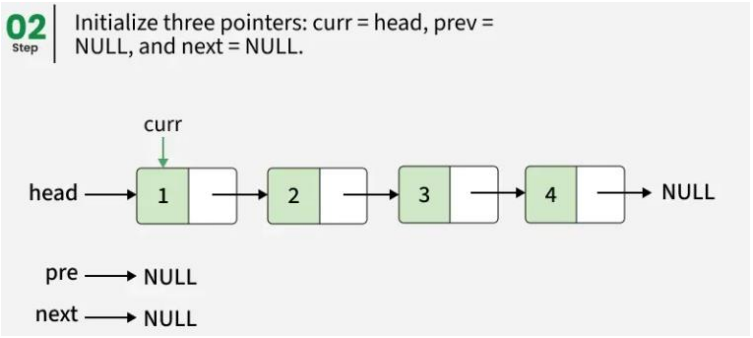
Iterative Method - O(n) Time and O(1) Space:

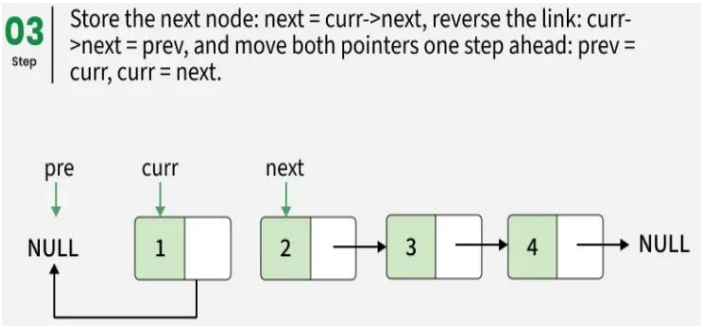
The idea is to reverse the linked list by changing the direction of links using three pointers: prev, curr, and next.

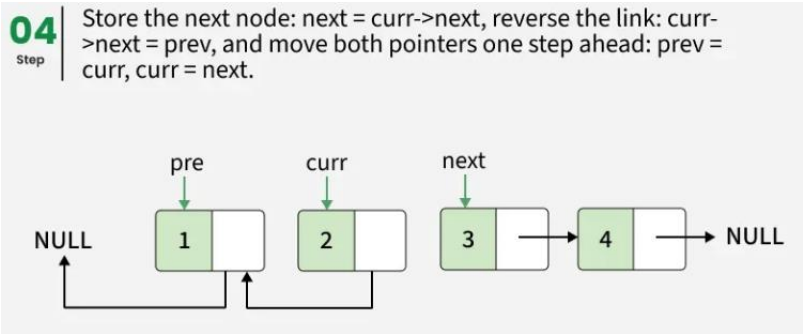
At each step,

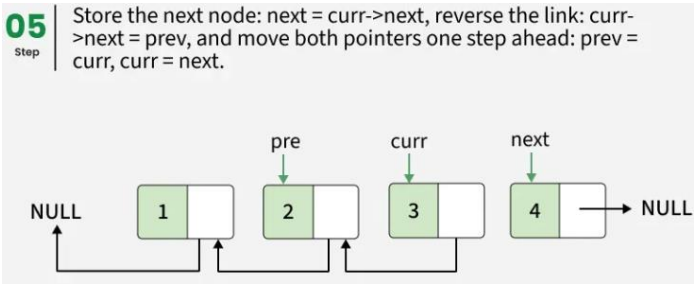
- *Point the current node to its previous node*
- *Then move all three pointers (prev, curr and next) forward until the list is fully reversed.*
- *At the end, prev becomes new head*

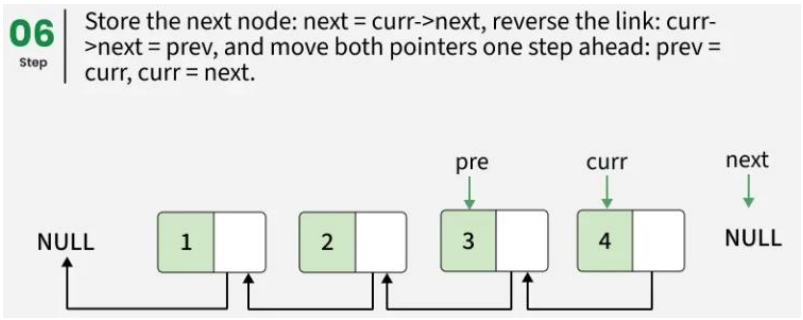


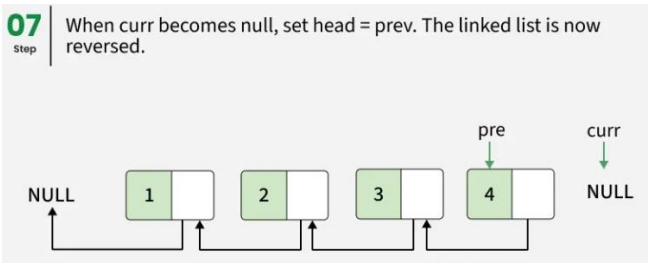


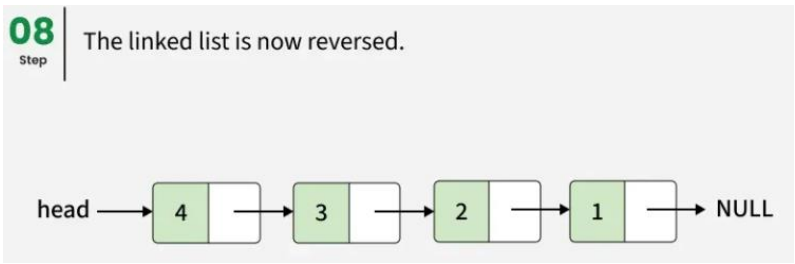












Implementation Code:

```
def reverseList(head):

    curr = head
    prev = None

    # traverse all the nodes of Linked List
    while curr is not None:

        # store next
        nextNode = curr.next

        # reverse current node's next pointer
        curr.next = prev

        # move pointers one position ahead
        prev = curr
        curr = nextNode

    return prev
```

Output: 5 -> 4 -> 3 -> 2 -> 1

Lab Question Codes

Singly Linked List (LLNode)

```python
class LLNode:
    def __init__(self, val):
        self.val = val
        self.next = None

    # ==========================
    # Checkpoint Methods
    # ==========================

    def length(self):
        """Return the number of nodes from this node to the end of the chain."""
        count = 0
        current = self
        while current is not None:
            count += 1
            current = current.next
        return count

    def contains(self, val):
        """Return True if val appears in this chain starting from this node, False otherwise."""
        current = self
        while current is not None:
            if current.val == val:
                return True
            current = current.next
        return False

    def insert_after(self, val):
        """Insert a new node with value val immediately after this node."""
        new_node = LLNode(val)
        new_node.next = self.next
        self.next = new_node

    def remove_next(self):
        """Remove the node immediately after this node. If none exists, do nothing."""
        if self.next is not None:
            self.next = self.next.next

    # ==========================
    # More Methods
    # ==========================

    def to_list(self):
        """Return a regular Python list containing values from this node to the end."""
        result = []
        current = self
        while current is not None:
            result.append(current.val)
            current = current.next
        return result

    def last(self):
        """Return the last node in the chain."""
        current = self
        while current.next is not None:
            current = current.next
        return current

    def append(self, val):
        """Add a new node to the end of the chain."""
        last_node = self.last()
```

```
        last_node.next = LLNode(val)

    def find(self, val):
        """Return the first node containing val, or None if val does not appear."""
        current = self
        while current is not None:
            if current.val == val:
                return current
            current = current.next
        return None

    def count(self, val):
        """Return the number of times val appears in the chain from this node onwards."""
        total = 0
        current = self
        while current is not None:
            if current.val == val:
                total += 1
            current = current.next
        return total

    def insert_after_value(self, target, val):
        """Find the first node containing target, then insert val immediately after it."""
        target_node = self.find(target)
        if target_node is not None:
            target_node.insert_after(val)

    def remove_next_after_value(self, target):
        """Find the first node containing target, then remove the node immediately after it."""
        target_node = self.find(target)
        if target_node is not None:
            target_node.remove_next()
```

Strech Doubly Linked List

```
class DLLNode:
    def __init__(self, val):
        self.val = val
        self.next = None
        self.prev = None

    def insert_after(self, val):
        """Insert a new DLLNode with value val immediately after this node."""
        new_node = DLLNode(val)
        old_next = self.next

        new_node.prev = self
        new_node.next = old_next

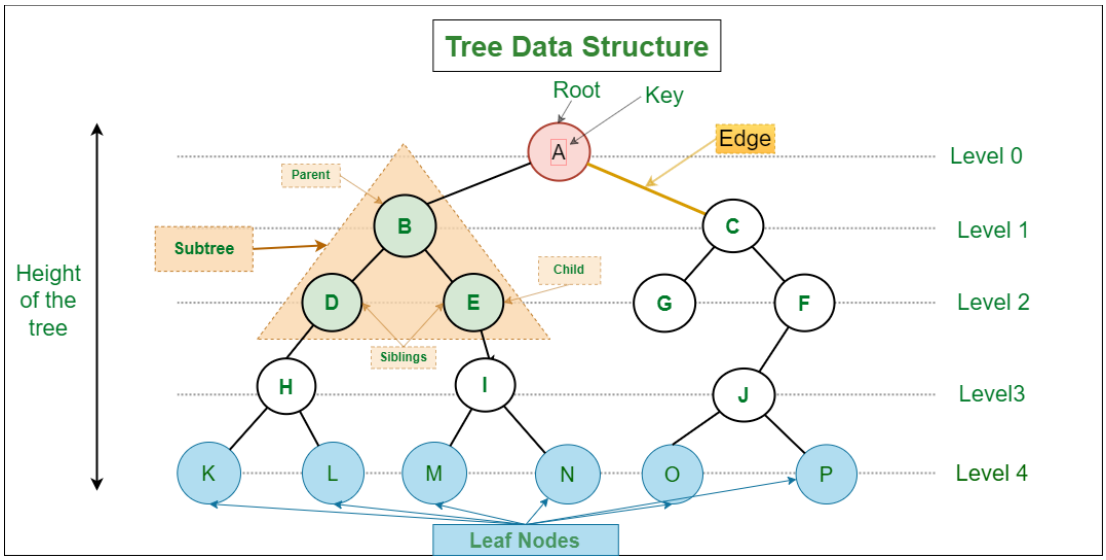
        self.next = new_node
        if old_next is not None:
            old_next.prev = new_node

    def remove_next(self):
        """Remove the node immediately after this node, updating both directions."""
        node_to_remove = self.next
        if node_to_remove is not None:
            node_after = node_to_remove.next
            self.next = node_after
            if node_after is not None:
                node_after.prev = self
```

```
    def to_list_backwards(self):
        """Traverse backward using prev references and return the values in a list."""
        result = []
        current = self
        while current is not None:
            result.append(current.val)
            current = current.prev
        return result
```

Week 6: Trees, Stacks, Queues

A tree is a non-linear data structure, or we can say that it is a type of hierarchical structure. It is like an inverted tree where roots are present at the top and leaf or child nodes are present at the bottom of the tree. The root node and its child nodes are connected through the edge like the branches in the tree.



Different Types of Tree Traversal Techniques

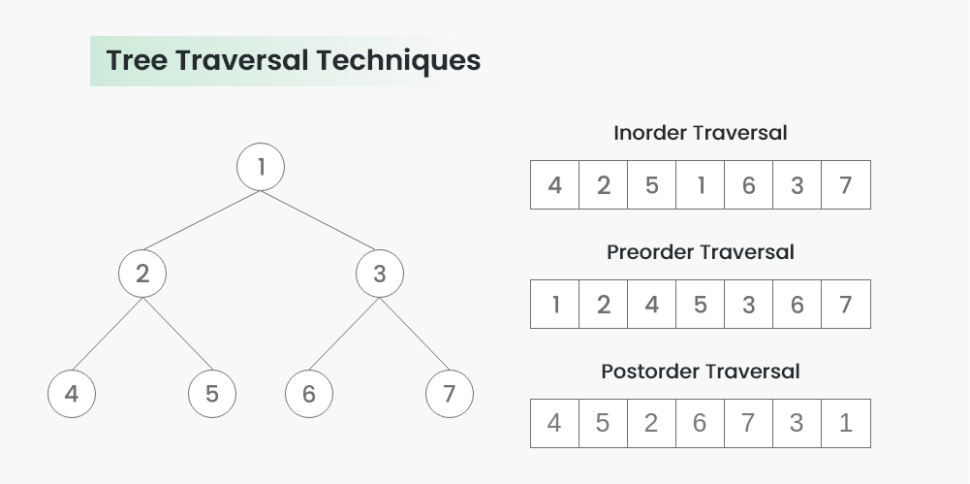
There are three types of traversal ways:

1. **Inorder**
2. **Preorder**
3. **Postorder**

Note: These traversal in trees are types of **depth first search (DFS)**

Overview of Tree Traversal Techniques

Below we see a binary tree and the result of each traversal technique.



Inorder Tree Traversal in Python

Inorder tree traversal is traversing **left → root → right node** of each **sub-tree**.

```
class TreeNode:

  def __init__(self, val=0, left=None, right=Null):
    self.val = val
    self.left = left
    self.right = right

def inorderTraversal(root):
  result = []  # Stores the final inorder traversal sequence (Left -> Root -> Right)
  stack = []  # Explicit stack to track nodes (replaces recursion call stack)
  current = root  # Pointer to track our current position in the tree

  # Continue as long as there are nodes to visit or nodes waiting in the stack
  while current or stack:

    # 1. Dive as far left as possible from the current node
    while current:
      stack.append(current)  # Save current node to stack before going left
      current = current.left  # Move to the left child

    # 2. Process the leftmost node (Root of the current subtree)
    current = stack.pop()  # Pop the last added node from the stack
    result.append(current.val)  # Visit/record the node's value

    # 3. Move to the right subtree to process it next
    current = current.right

  return result
```

Time Complexity: O(N) because every node in the tree is visited exactly once.

Example Usage

```
# Constructing a simple binary tree:
#       1
#        \
#         2
#        /
#       3
root = TreeNode(1)
root.right = TreeNode(2)
root.right.left = TreeNode(3)
print(inorderTraversal(root))  # Output: [1, 3, 2]
```

Preorder Tree Traversal in Python

Pre-order tree traversal is basically traversing the **Root → Left → Right node** of each sub-tree.

```
class TreeNode:

  def __init__(self, val=0, left=None, right=None):
    self.val = val
    self.left = left
    self.right = right

def preorderTraversal(root):
  # Return an empty list if the tree is empty
  if not root:
    return []

  result = []  # Stores the final preorder sequence (Root -> Left -> Right)
  stack = [root]  # Initialize the stack with the root node

  # Continue until all nodes in the stack are processed
  while stack:
    # 1. Pop the top node (this acts as our Root)
    current = stack.pop()
    result.append(current.val)  # Visit/record the node's value

    # 2. Push right child first, then left child
    # (Because it's a LIFO stack, pushing right first ensures left is processed first)
    if current.right:
      stack.append(current.right)
    if current.left:
      stack.append(current.left)

  return result
```

Why push the Right child before the Left Child?

- Because a stack operates on a **LIFO** (Last-in, First-In)
- Therefore, we must push the Left child last, so it sits on top of the stack. We push the Right child first, so it sits underneath.

Example Usage

```
# Constructing the following binary tree:
#        1
#       / \
#      2   3
#     / \
#    4   5
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)

# Run the traversal
output = preorderTraversal(root)
print(output)
```

Output: [1, 2, 4, 5, 3]

How the Preorder Sequence works:

1. `1` (Root 1 (Root of the entire tree)v
2. `2` (Root of the left subtree)
3. `4` (Left-most leaf of node 2)
4. `5` (Right child of node 2)
5. `3` (Root of the right subtree)

Stack & Queue

Stack

A **Stack** is a linear data structure that follows the **LIFO (Last-In, First-Out)** principle: the last element added is the first one removed. Think of a physical stack of plates—you place new plates on top, and you remove plates from the top.

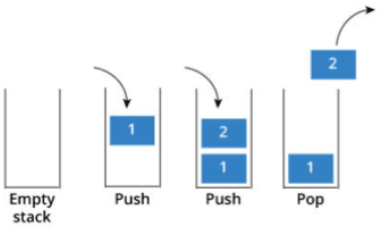
- **Primary Operations:**
  - `push(x)`: Add an element to the top ($O(1)$).
  - `pop()`: Remove and return the top element ($O(1)$).
  - `peek() / top()`: Inspect the top element without removing it ($O(1)$).
  - `isEmpty()`: Check if the stack contains zero elements ($O(1)$).
- **Common Applications:** Undo/redo history in editors, parsing syntax/brackets, evaluating postfix expressions, and simulating iterative call stacks.

Queue

A standard **Queue** follows the **FIFO (First-In, First-Out)** principle: the first element added is the first one processed. Think of a checkout line at a store—new arrivals join at the back, and service happens at the front.

- **Primary Operations:**
  - enqueue(x): Insert an element at the rear/back ($O(1)$).
  - dequeue(): Remove and return the element at the front ($O(1)$).
  - front() / peek(): View the front element without removal ($O(1)$).
  - isEmpty(): Check if the queue is empty ($O(1)$).
- **Common Applications:** CPU task scheduling, print job buffering, asynchronous message brokers, and Breadth-First Search (BFS).

Data Structure Basics





Deque (Double-Ended Queue)

A **Deque** (pronounced *"deck"*) generalizes both stacks and queues by allowing insertion and removal at **both** ends.

- **Primary Operations (all $O(1)$ amortized/worst-case depending on backing structure):**
  - `append(x)` / `push_back(x)`: Insert at the rear.
  - `appendleft(x)` / `push_front(x)`: Insert at the front.
  - `pop()` / `pop_back()`: Remove from the rear.
  - `popleft()` / `pop_front()`: Remove from the front.
- **Common Applications:** Sliding window maximum/minimum algorithms, work-stealing job schedulers, and maintaining browser forward/back histories.

Queue Implementations

A queue interface can be backed by different underlying data structures, each carrying distinct performance characteristics.

A. Dynamic Array / List-Based Queue

- **Mechanism:** Elements are appended to a dynamic array (`list.append(x)`). Removal uses `list.pop(0)`.
- **The Problem:** While adding to the end is $O(1)$ amortized, removing from index 0 forces every remaining element in memory to shift one position left.
- **Complexity:**
  - Enqueue: $O(1)$ amortized
  - Dequeue: $O(n)$ — **inefficient for large workloads**

B. Deque-Based Queue

- **Mechanism:** In Python (collections.deque) or C++ (std::deque), deques are internally structured as doubly linked blocks/chunks of fixed-size arrays.
- **Advantages:** Avoids shifting elements when popping from index 0. Pointers track both ends across allocated memory blocks.
- **Complexity:**
  - Enqueue (append): $O(1)$
  - Dequeue (popleft): $O(1)$
- **Trade-off:** Minimal memory overhead for block pointers, but optimal for FIFO queues in high-throughput applications.

C. Linked-List-Based Queue

- **Mechanism:** Maintain a singly linked list with two explicit pointers: head (front) and tail (rear).
  - **Enqueue:** Create a new node, point tail.next to it, and move tail forward ($O(1)$).
  - **Dequeue:** Read head.data, advance head = head.next, and clean up ($O(1)$).
- **Complexity:**
  - Enqueue: $O(1)$
  - Dequeue: $O(1)$
- **Trade-off:** Avoids dynamic resizing spikes, but incurs memory overhead per node for pointers and suffers from poorer cache locality compared to contiguous arrays.

Breadth-First Search (BFS)

Breadth-First Search explores a graph or tree **level by level**, visiting all direct neighbors at depth d before moving to depth $d+1$.

Because exploration is strictly ordered by discovery time, BFS **fundamentally relies on a FIFO queue** rather than recursion (which implicitly builds a LIFO call stack and produces Depth-First Search).

Iterative Algorithm

1. Enqueue the start node into a Queue.
2. Mark the start node as visited (using a set or boolean array to avoid cycles).
3. While the queue is not empty:
   - Dequeue the front node u.
   - Process u.
   - For every adjacent neighbor v of u:
     - If v has not been visited:
       - Mark v as visited.
       - Enqueue v.

Code Implementation:

```
from collections import deque

def bfs(graph, start_node):
    visited = set([start_node])
    queue = deque([start_node])

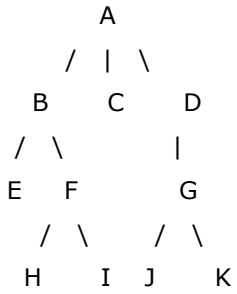
    while queue:
        current = queue.popleft()
        print(f"Visited: {current}")

        for neighbor in graph.get(current, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

- **Complexity:**
  - Time: $O(V+E)$ where V is vertices and E is edges.
  - Space: $O(V)$ for the queue and visited set.
- **Key Property:** In an unweighted graph, the first time BFS reaches any node, it has found the **shortest path** (fewest edges) from the source.

Lab Questions

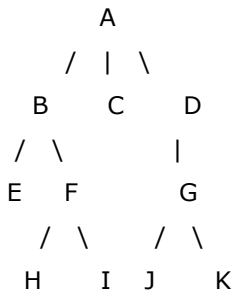
Trace DFS



In valuesDFS, the stack uses LIFO (last-in, first-out) order, pushing children in reverse so the leftmost child pops first. Stack contents are represented with the top of the stack on the right (index -1).

Iteration	**Node Removed**	**Stack (Top on right)**	**Result**
Initial	—	[A]	[]
1	A	[D, C, B]	['A']
2	B	[D, C, F, E]	['A', 'B']
3	E	[D, C, F]	['A', 'B', 'E']
4	F	[D, C, I, H]	['A', 'B', 'E', 'F']
5	H	[D, C, I]	['A', 'B', 'E', 'F', 'H']
6	I	[D, C]	['A', 'B', 'E', 'F', 'H', 'I']
7	C	[D]	['A', 'B', 'E', 'F', 'H', 'I', 'C']
8	D	[G]	['A', 'B', 'E', 'F', 'H', 'I', 'C', 'D']
9	G	[K, J]	['A', 'B', 'E', 'F', 'H', 'I', 'C', 'D', 'G']
10	J	[K]	['A', 'B', 'E', 'F', 'H', 'I', 'C', 'D', 'G', 'J']
11	K	[]	['A', 'B', 'E', 'F', 'H', 'I', 'C', 'D', 'G', 'J', 'K']

Trace BFS



In valuesBFS, nodes are popped from the left (FIFO) and children are added to the right

Iteration	Node Removed	Queue (Front on left)	Result
Initial	—	[A]	[]
1	A	[B, C, D]	['A']
2	B	[C, D, E, F]	['A', 'B']
3	C	[D, E, F]	['A', 'B', 'C']
4	D	[E, F, G]	['A', 'B', 'C', 'D']
5	E	[F, G]	['A', 'B', 'C', 'D', 'E']
6	F	[G, H, I]	['A', 'B', 'C', 'D', 'E', 'F']
7	G	[H, I, J, K]	['A', 'B', 'C', 'D', 'E', 'F', 'G']
8	H	[I, J, K]	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
9	I	[J, K]	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']
10	J	[K]	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
11	K	[]	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K']

Here is the complete `TreeNode` implementation with detailed comments added to explain how each method functions without recursion:

```
from collections import deque


class TreeNode:

    def __init__(self, val, children=None):
        self.val = val
        # If no children are passed, default to an empty list; otherwise use provided list[cite: 4]
        self.children = [] if children is None else children



    # 1. Trace DFS (Depth-First Search)
    def valuesDFS(self):
        result = []
        # The stack holds the nodes we still need to visit (LIFO: Last-In, First-Out)
        stack = [self]

        while stack:
            # Pop the most recently added node from the end of the list
            node = stack.pop()
            result.append(node.val)

            # We iterate through children in reverse so that the first child
            # ends up at the very top of the stack and gets visited next[cite: 4]
            for child in reversed(node.children):
                stack.append(child)

        return result



    # 2. Trace BFS (Breadth-First Search)
    def valuesBFS(self):
        result = []
        # A double-ended queue processes nodes level-by-level (FIFO: First-In, First-Out)[cite: 4]
        queue = deque([self])

        while queue:
            # Remove the oldest node waiting at the front of the queue
            node = queue.popleft()
            result.append(node.val)

            # Enqueue all children from left to right to be visited after current level[cite: 4]
            for child in node.children:
                queue.append(child)

        return result
```

```python
# 3. Count leaves
def countLeaves(self):
    count = 0
    queue = deque([self])

    while queue:
        node = queue.popleft()

        # A leaf node is defined as having no children
        if not node.children:
            count += 1
        else:
            # If it has children, add them to queue to check them later
            for child in node.children:
                queue.append(child)

    return count



# 4. Find a value
def find(self, val):
    # We can use a standard traversal to scan nodes until we find a match
    queue = deque([self])

    while queue:
        node = queue.popleft()

        # Return the node object immediately upon finding the target value
        if node.val == val:
            return node

        for child in node.children:
            queue.append(child)

    # val was never found anywhere in the subtree
    return None



# 5. Find the shallowest match
def findShallowest(self, val):
    # BFS explores node-by-node level-by-level (closest depths first).
    # Therefore, the very first match hit in BFS is guaranteed to be at the minimum depth.
    queue = deque([self])

    while queue:
        node = queue.popleft()

        if node.val == val:
            return node

        for child in node.children:
            queue.append(child)

    return None
```

```python
# 6. Values at a particular depth
def valuesAtDepth(self, target_depth):
    if target_depth < 0:
        return []

    result = []
    # Store a tuple of (node, current_depth) to track depth level explicitly[cite: 4]
    queue = deque([(self, 0)])

    while queue:
        node, depth = queue.popleft()

        # Collect value if the node is at the requested depth
        if depth == target_depth:
            result.append(node.val)
        # Only expand children if we haven't reached target depth yet (prunes deeper nodes)
        elif depth < target_depth:
            for child in node.children:
                queue.append((child, depth + 1))

    return result
```

```python
# 7. Height
def height(self):
    max_depth = 0
    # Track depth from self (depth 0) to find maximum depth reached
    queue = deque([(self, 0)])

    while queue:
        node, depth = queue.popleft()

        # Update highest depth observed so far
        if depth > max_depth:
            max_depth = depth

        # Add child nodes with depth incremented by 1
        for child in node.children:
            queue.append((child, depth + 1))

    return max_depth
```

```
# ==========================
# Stretch Methods
# ==========================

# 8. Count nodes at each depth
def countAtEachDepth(self):
    counts = []
    queue = deque([(self, 0)])

    while queue:
        node, depth = queue.popleft()

        # If encountering a new depth level for the first time, append a new counter
        if depth == len(counts):
            counts.append(1)
        else:
            # Increment tally for this existing depth
            counts[depth] += 1

        for child in node.children:
            queue.append((child, depth + 1))

    return counts




# 9. Find a path
def pathTo(self, val):
    # Store tuples of (current_node, list_of_values_accumulated_so_far)
    queue = deque([(self, [self.val])])

    while queue:
        node, path = queue.popleft()

        # Match found: return the full path list from root to this node
        if node.val == val:
            return path

        # Extend the path with each child's value and enqueue
        for child in node.children:
            queue.append((child, path + [child.val]))

    return None
```